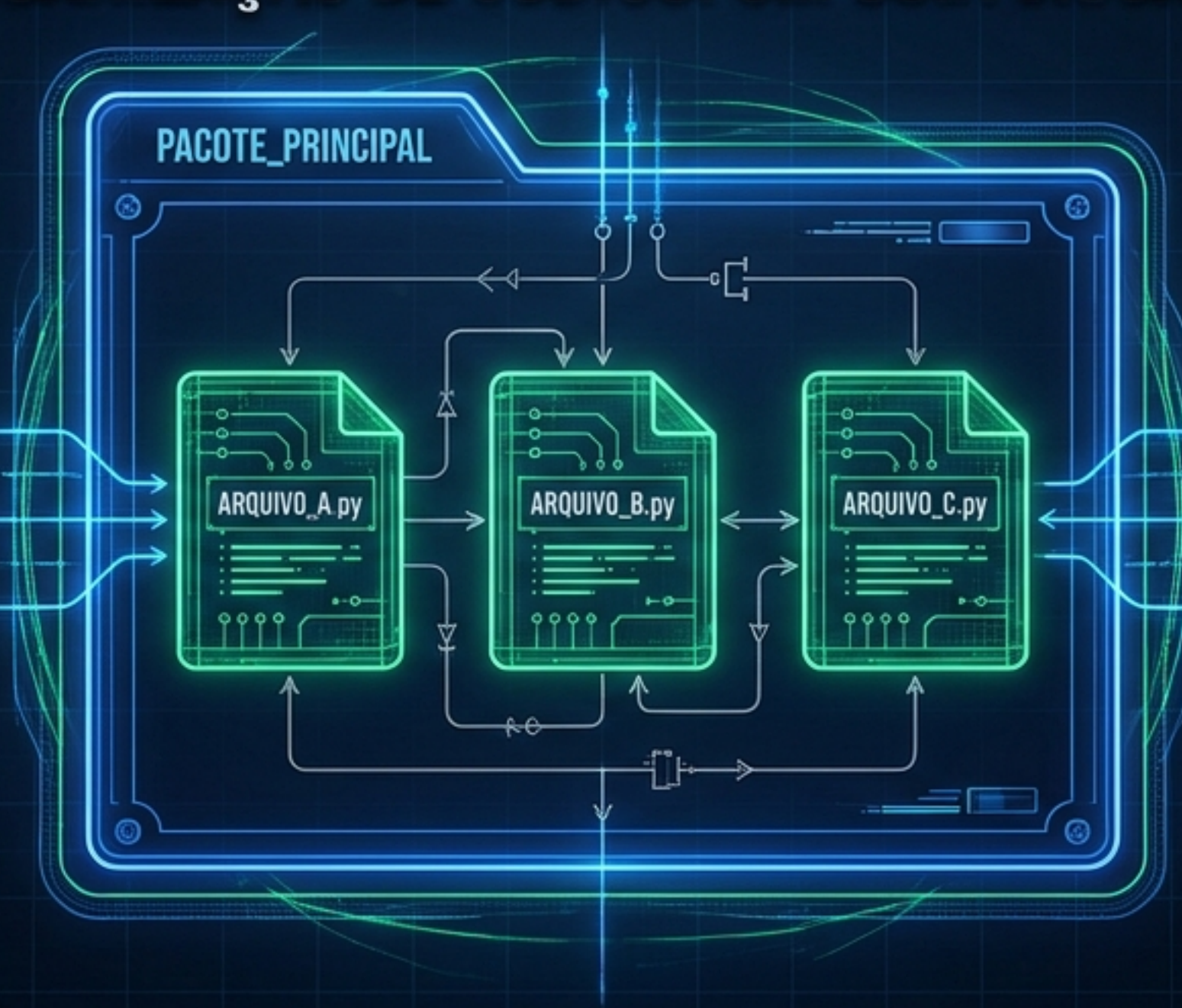


MÓDULOS E PACOTES EM PYTHON

ARQUITECTURA E ORGANIZAÇÃO DE CÓDIGO: UM GUIA VISUAL PARA ESTUDANTES



DO ARQUIVO ÚNICO À ENGENHARIA DE SOFTWARE ESCALÁVEL

Sumário da Arquitetura



1. O Conceito de Módulo



2. Importação e Namespaces



3. O Caminho de Busca (Path)



4. Scripts vs. Módulos



5. Empacotamento (Packages)



6. Resumo e Sintonia Final

Por que modularizar?



Simplicidade

Foco em subconjuntos do problema.



Manutenção

Fronteiras lógicas claras para alterações.



Testes

Teste peças isoladas antes da integração.



Reuso

Defina uma vez, use em múltiplos projetos.



Escopo

Evita colisões de nomes (namespaces únicos).

Anatomia de um Módulo Python

utils.py

****Docstring****

Documentação e metadados.

****Código Executável****

Roda automaticamente na primeira importação.

```
"""  
Este é o docstring do módulo.  
Ele fornece uma descrição de alto nível do propósito do módulo.  
"""
```

```
print("Este código roda automaticamente na importação!")
```

```
def printer(some_object):  
    """Uma função utilitária."""  
    print(some_object)
```

```
class Shape:  
    """Uma classe base para formas."""  
    pass
```

```
default_shape = Shape()
```

****Funções****

Lógica reutilizável.

****Classes****

Definições de objetos.

****Variáveis****

Dados pré-definidos.

Conectando Arquivos: O Comando `import`

Um módulo não é automaticamente acessível. Ele precisa ser importado.

my_app.py

```
import utils  
utils.printer(utils.default_shape)
```

utils.py

```
def printer(some_object):  
    class Shape:
```

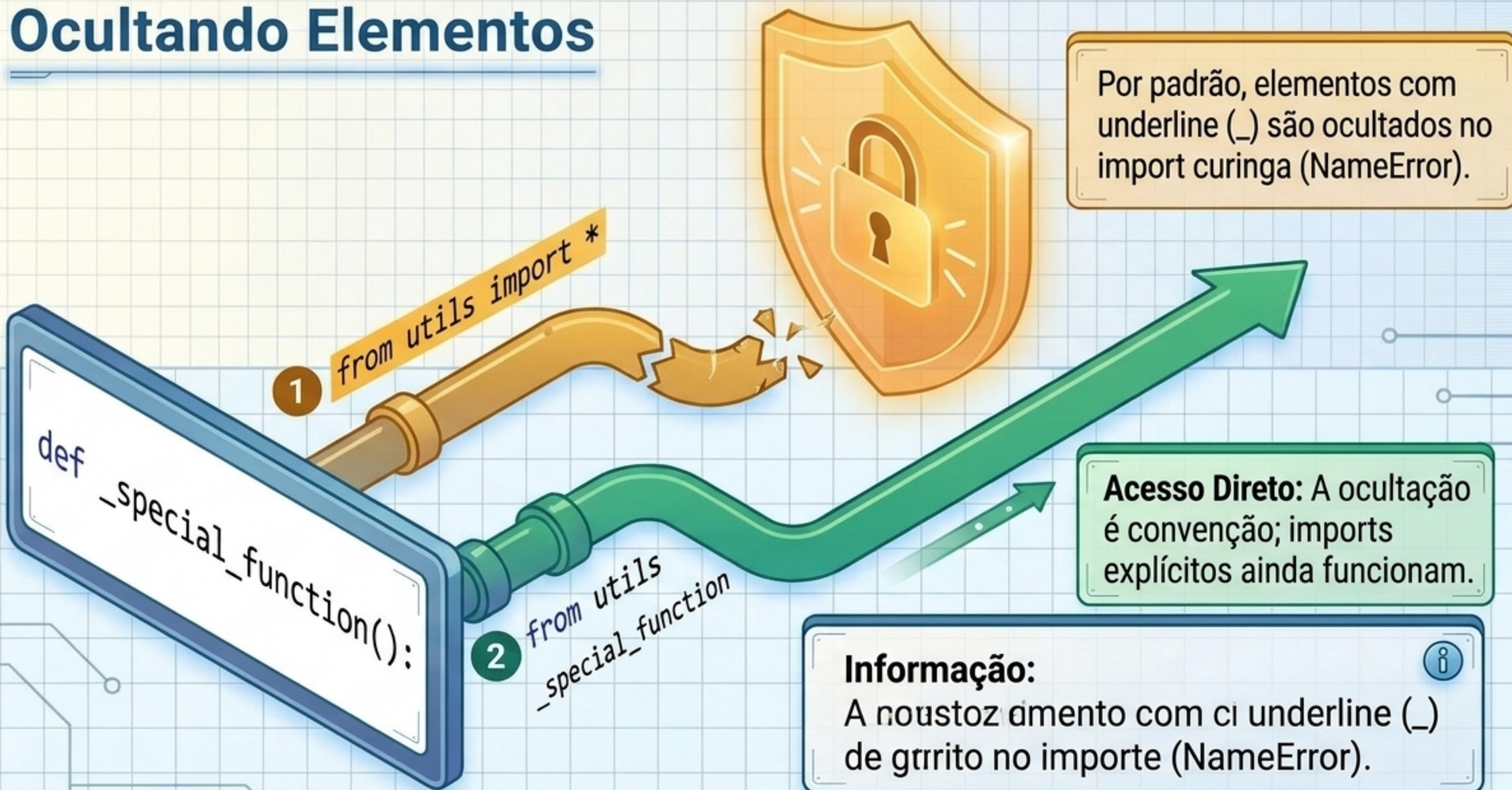
Regra de Ouro:
Não incluímos a extensão
.py no import.

Namespace:
O conteúdo deve ser prefixado com o
nome do módulo para evitar conflitos.

Matriz de Estilos de Importação

Sintaxe	Impacto no Namespace	Exemplo Prático
<code>import modulo</code>	Exige prefixo completo (Mais seguro)	<code>utils.printer()</code>
<code>from modulo import item</code>	Remove necessidade do prefixo (Cuidado com colisões)	<code>s = Shape('rectangle')</code>
 <code>from modulo import *</code>	Importa tudo (Padrão perigoso - Wildcard)	Traz tudo para o escopo global.
<code>import modulo as alias</code>	Renomeia o módulo ou função no escopo local	<code>import utils as u</code> <code>u.printer()</code>

Controle de Acesso: Ocultando Elementos



Raio-X do Módulo:

Propriedades Embutidas

Todo módulo possui propriedades especiais (dunder) que revelam sua identidade.

Properties Panel

`__name__`: O nome oficial do módulo.

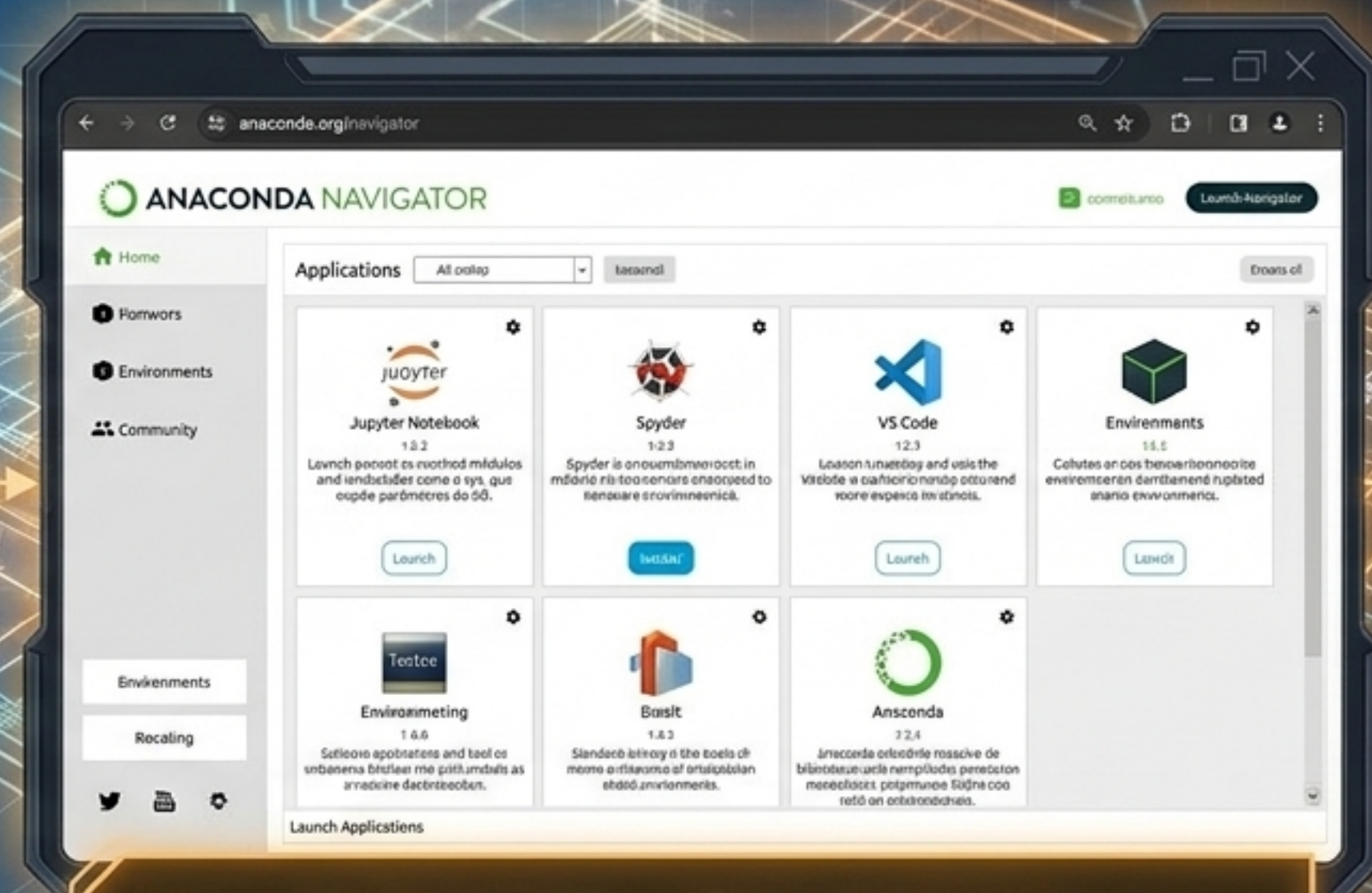
`__doc__`: A docstring (documentação).

`__file__`: O caminho do arquivo físico.

```
>>> dir()
['__builtins__',
 '__doc__',
 '__file__',
 '__loader__',
 '__name__',
 '__package__',
 'printer',
 'Shape',
 'done']
```

A ferramenta `dir()` lista todo o conteúdo do módulo.

O Ecossistema: Built-in e Terceiros



Standard Library: Módulos embutidos como o sys, que expõe parâmetros do SO.

Anaconda: Repositório massivo de bibliotecas pré-compiladas para Ciência de Dados.

O Caminho de Busca (Path): Como o Python acha os arquivos?

Alerta de Shadowing: Nomear seu arquivo com o nome de um módulo do sistema (ex: **math.py**) faz o Python achá-lo no Passo 1, quebrando o código original.

1. Diretório Atual

2. Variável **PYTHONPATH**

3. Caminho Padrão do Sistema

A Dupla Natureza: Script vs. Módulo



Execução Direta

```
python module1.py
```

```
module1.py: __name__ is '__main__'
```

O arquivo é o programa principal. A propriedade **`__name__`** vira **`'__main__'`**.



Sendo Importado

```
import module1
```

```
importing module1...  
__name__ is 'module1'
```

Age como **biblioteca**. A propriedade **`__name__`** mantém o nome do módulo.

0 Idioma Pythonic: `if __name__ == '__main__':`

```
def main():  
    x = 1 + 2  
    print('x is', x)  
    f1()  
    f2()
```

```
if __name__ == '__main__':  
    main()
```

Isola a execução em uma função principal.

O guardião. Testa se foi executado diretamente. Se importado, o bloco é ignorado, impedindo a execução de códigos de teste soltos.

Escalando a Arquitetura: O que são Pacotes?

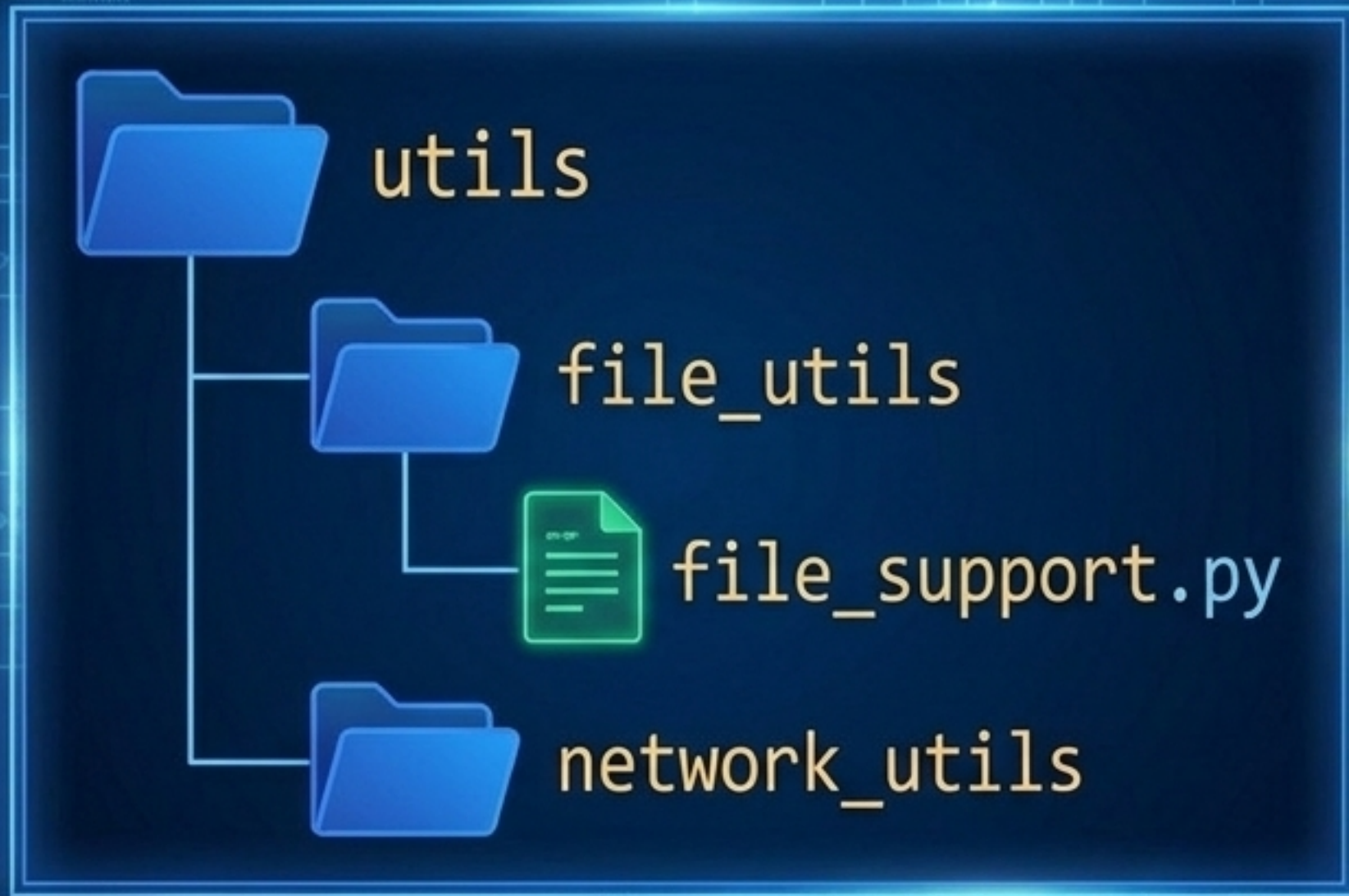
Um pacote é um diretório do sistema operacional contendo um ou mais módulos.

O Arquivo Chave:
`__init__.py`.

O código dentro deste arquivo é executado apenas uma vez, na primeira referência ao pacote.



Navegando em Árvores Complexas: Subpacotes



Sintaxe de Ponto (Dot-Notation):
Espelha a barra (/) do sistema operacional.

```
import utils.file_utils.file_support
```

Controle de Exportação (`__all__`):
Limita quais módulos são exportados em um wildcard import no nível do pacote.

Blueprint Final: Resumo da Arquitetura

Organização Física

- **Módulo:** 1 Arquivo (.py).
- **Pacote:** 1 Diretório. Requer `__init__.py`.
- **Subpacote:** Acessado via `pacote.subpacote`.

Sintaxe de Importação

- `import X`
- `from X import Y`
- `import X as Z`
- `_oculto`

Regras de Execução

- `sys.path` (Current -> PYTHONPATH -> OS)
- `if __name__ == '__main__':` protege o script.